VULNERABILITY ASSESSMENT & PROFESSIONAL REPORT

SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

PortSwigger Web Security Academy Lab Environment

Prepared by: Taraka Divya Ketha
Prepared for: Cryptonic Area Team
Date: July 24, 2026
Report Classification: Confidential
Assessment Type: Authorized Web Application Security Testing

1. Executive Summary

During an authorized security assessment of the target application, a SQL Injection vulnerability was identified in the product category filtering feature. This vulnerability allowed unauthorized access to data that the application was designed to keep hidden from standard users, including unreleased product records.

The vulnerability arises from a failure to properly separate user input from executable database commands, a well-known and highly preventable class of security weakness. If left unaddressed in a production environment, this type of flaw could expose sensitive business or customer data, and in more severe cases, could allow an attacker to manipulate or extract data beyond the scope observed in this assessment.

It is recommended that the development team prioritize remediation of this finding by adopting parameterized queries across all database-facing input fields, as detailed in Section 5 of this report. Given the ease of exploitation and the sensitivity of data potentially at risk, this finding should be treated as High severity and addressed before the application is deployed to a production environment.

2. Scope & Methodology

This assessment was limited to the product category filtering functionality of the target lab application, provided by PortSwigger Web Security Academy for authorized security testing purposes. Testing was conducted manually using Burp Suite (Proxy and Repeater) and Mozilla Firefox to identify and validate a SQL Injection vulnerability in the application's WHERE clause query logic. No other application features, endpoints, or infrastructure components were assessed, and no automated scanning tools were used as part of this engagement.

References: PortSwigger Web Security Academy — *SQL injection vulnerability in WHERE clause allowing retrieval of hidden data*: https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data

3. Vulnerability Analysis

3.1 Vulnerability Classification

The application is affected by a SQL Injection vulnerability (CWE-89) located in the category filtering functionality of the product listing page. User-supplied input submitted via the category parameter is incorporated directly into a SQL query without sanitization, validation, or the use of parameterized statements, permitting an attacker to alter the logical structure of the executed query.

The application's intended query construction is as follows:

```
SELECT * FROM products WHERE category = 'Pets' AND released = 1
```

By submitting the following value as the category parameter:

```
Pets' OR 1=1--
```

The query executed by the backend is transformed into:

```
SELECT * FROM products WHERE category = 'Pets' OR 1=1--' AND released = 1
```

The inline comment sequence (--) causes the database to disregard the remainder of the statement, effectively nullifying the released = 1 condition. As the clause 1=1 evaluates as true for every row, the query returns the complete contents of the products table irrespective of category or release status.

3.2 Root Cause

The vulnerability stems from improper handling of user input at the data access layer. The application concatenates untrusted input directly into a SQL query string rather than treating it as parameterized data. This failure to enforce a boundary between executable code and user-supplied data is the underlying cause of SQL Injection vulnerabilities in general and is the specific cause of the behavior observed in this instance.

3.3 Impact Assessment

Exploitation of this vulnerability enables unauthorized disclosure of data that the application is designed to restrict — in this case, product records marked as unreleased. While the demonstrated impact is limited to the disclosure of hidden product listings, the presence of unsanitized query construction indicates a systemic weakness that, in a production environment, could plausibly be leveraged to:

- Enumerate or extract data from additional database tables, including user credentials or personal data
- Modify or delete records within the affected database
- Achieve broader compromise of the database, contingent on the privilege level of the application's database account

The business impact of this class of vulnerability typically includes data confidentiality breaches, regulatory non-compliance (e.g., GDPR, PCI-DSS, depending on data classification), and reputational harm resulting from disclosure.

3.4 Affected Assets

Asset	Exposure
Application database	Direct exposure via unsanitized query execution
End users	Indirect risk if similar query patterns exist in authentication, search, or account-management functions
Business operations	Regulatory and reputational risk in the event of exploitation on a production system

3.5 CVSS Score

Base Score: 7.5 (High)

```
CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
```

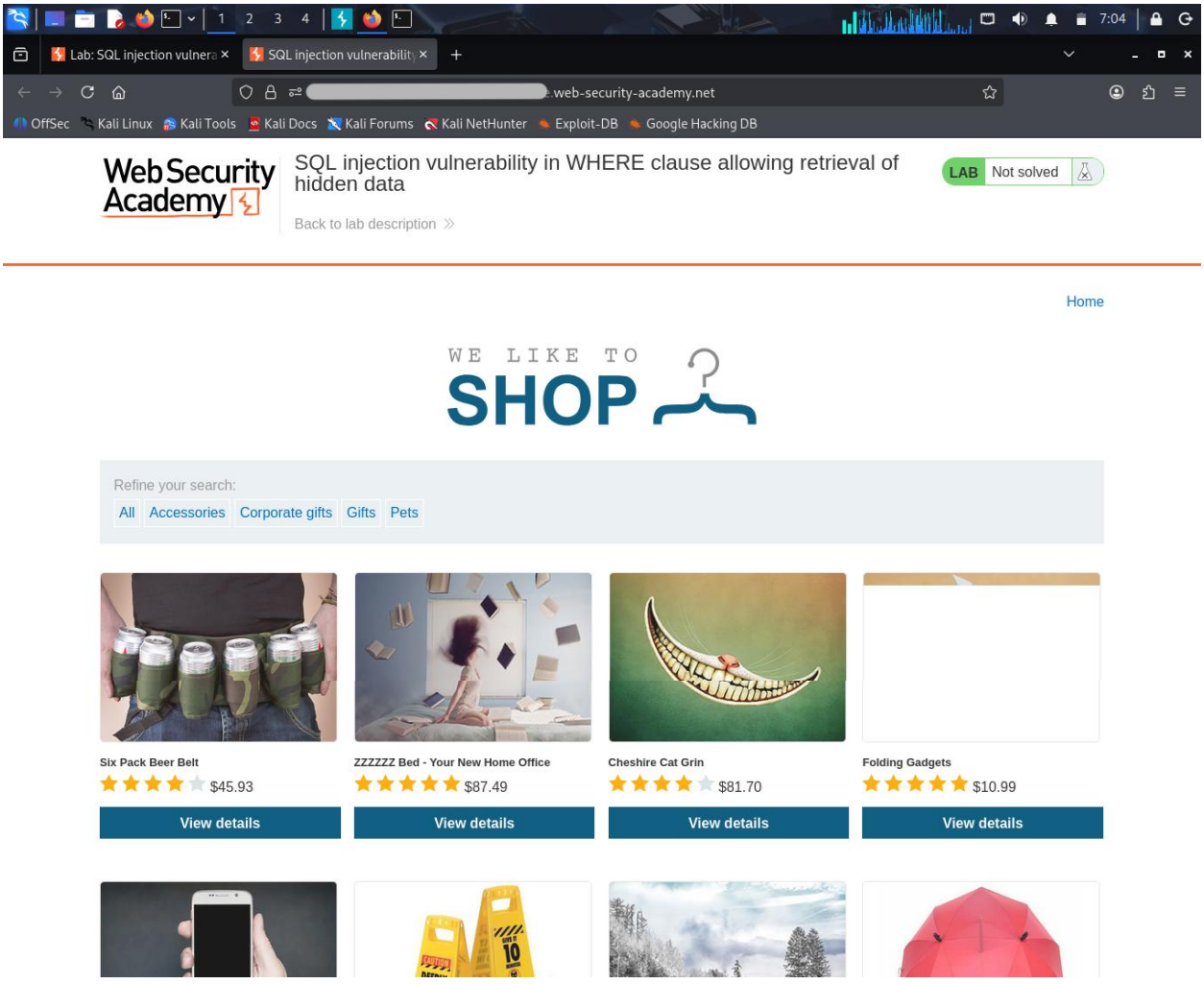
4. Proof of Concept

4.1 Testing Environment

Item	Detail
Target	PortSwigger Web Security Academy — SQL injection vulnerability in WHERE clause allowing retrieval of hidden data
Tools	Burp Suite (Proxy, Repeater), Mozilla Firefox
Authorization	Testing conducted within a PortSwigger-provided lab environment intended for authorized security research

4.2 Reproduction Steps

Step 1: The application's product listing page was loaded in its default, unmodified state, prior to any category selection or request tampering. This establishes a visual baseline of normal application behavior, confirming that only released products are displayed to a standard user under ordinary conditions.



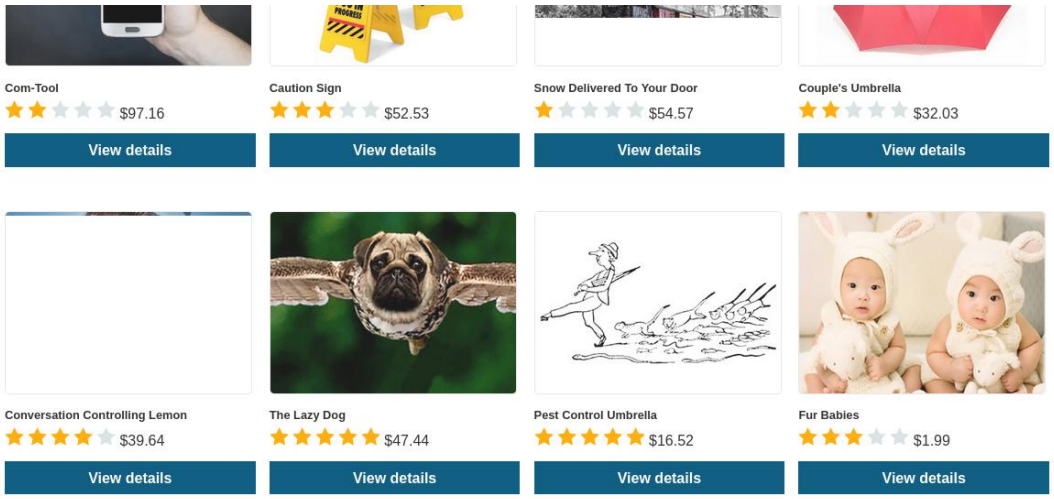


Figure 1: Default product listing page prior to exploitation, showing only released products.

Step 2: A baseline request was captured by selecting the "Pets" product category and intercepting the resulting request in Burp Suite.

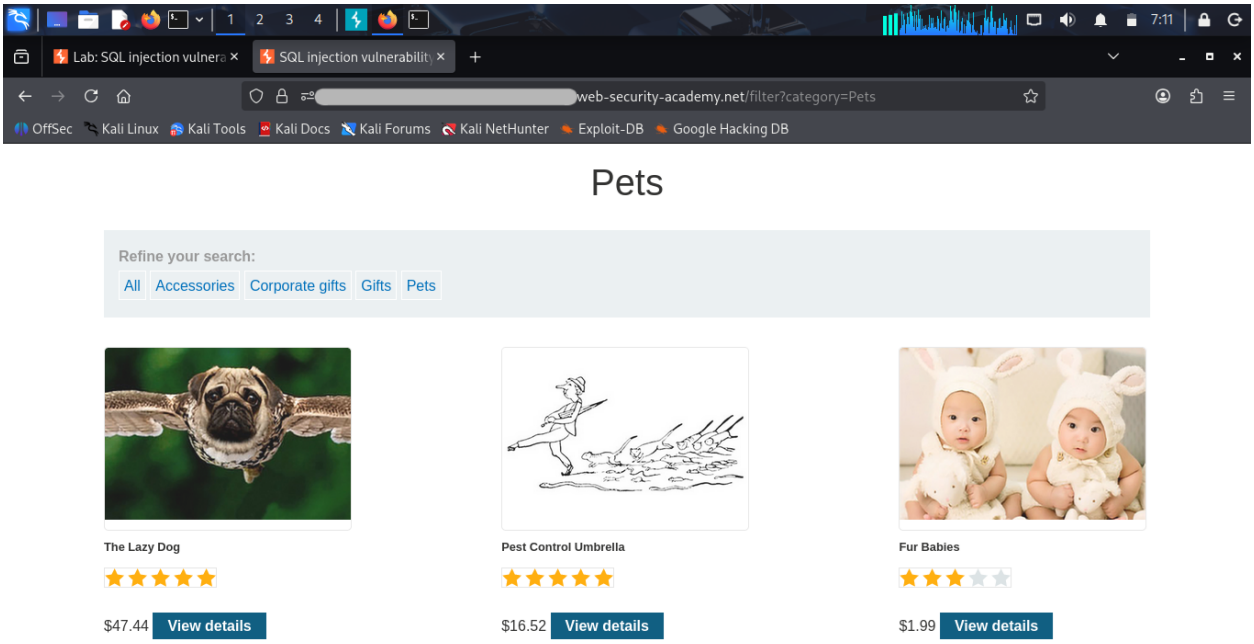


Figure 2: Baseline request for category parameter "Pets".

Step 3: The intercepted request was forwarded to Burp Repeater for manual modification.

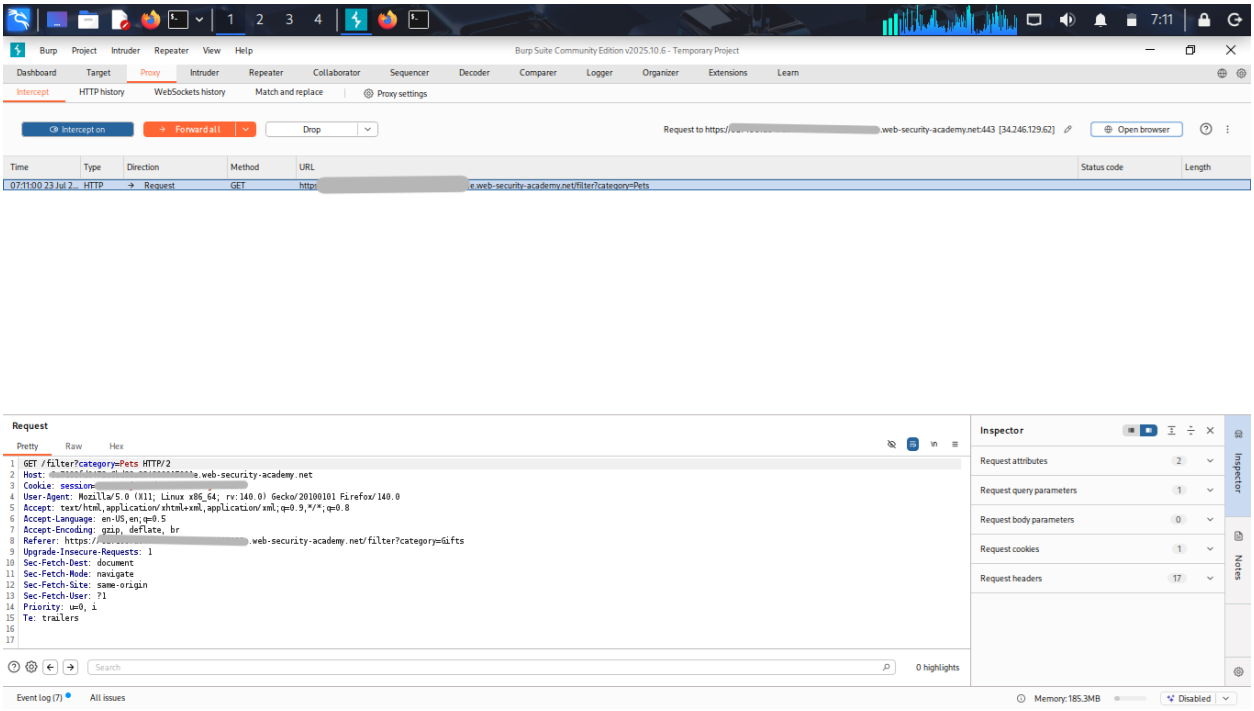


Figure 3: Intercepted request in Burpsuite is showing.

Step 4: The category parameter value was replaced with the payload Pets' OR 1=1--.

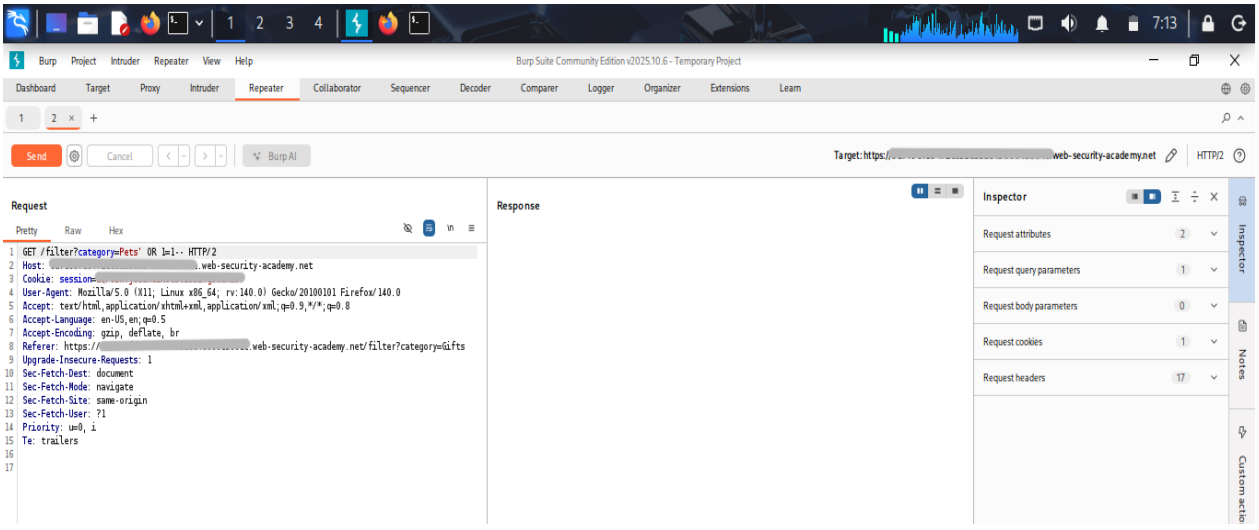


Figure 4: Modified request in Repeater with the injected payload.

Step 5: The modified request was transmitted, and the server response was recorded.

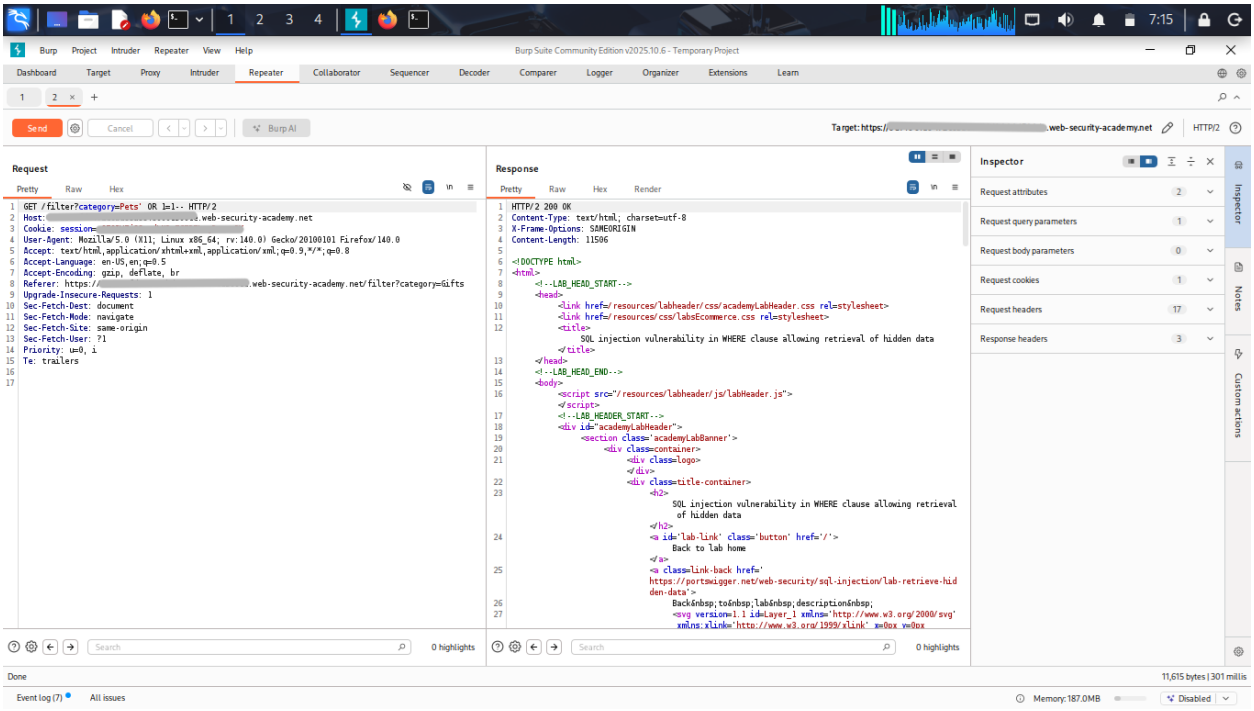
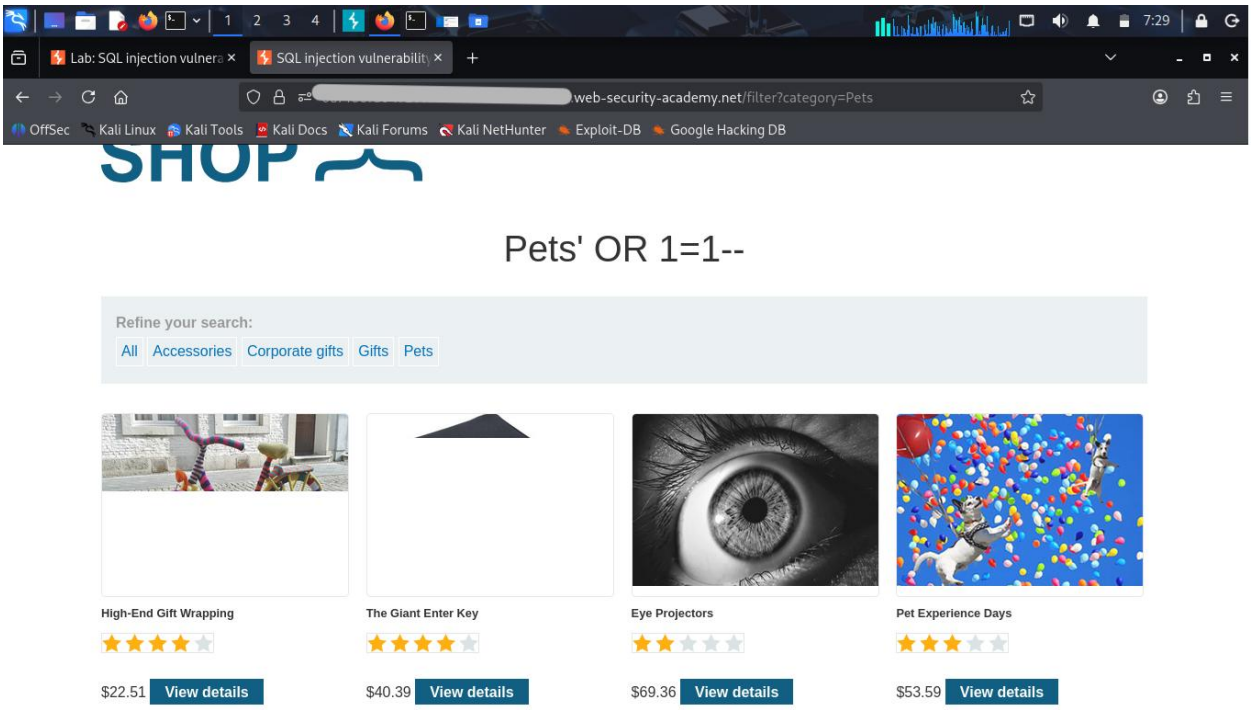


Figure 5: Raw response showing additional product records not present in the baseline response.

Step 6: The result was independently verified by reloading the corresponding request in the browser.



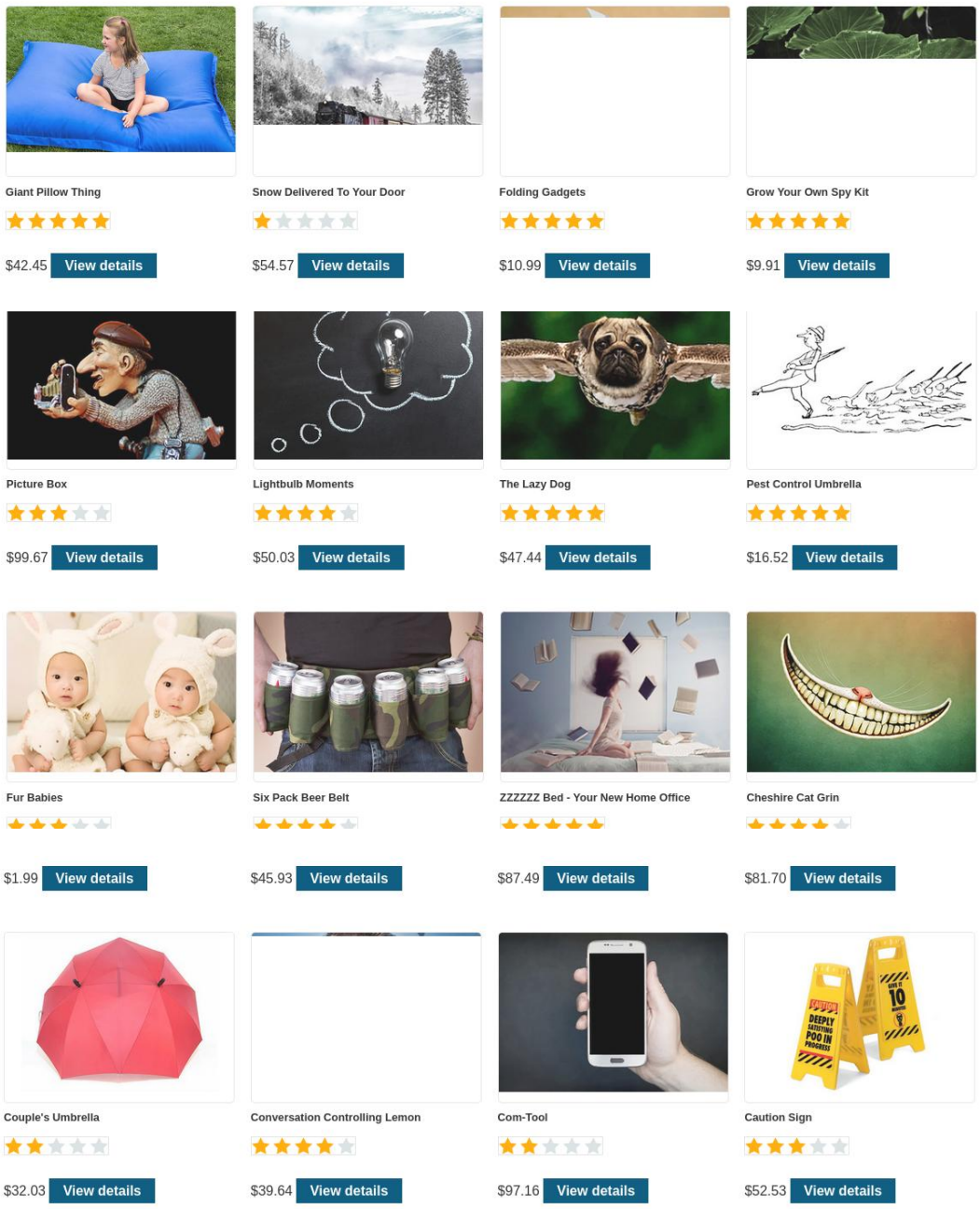


Figure 6: Full response page, top section — unreleased products visible in the storefront.

Step 7: The lab status was confirmed as "Solved," verifying successful exploitation.

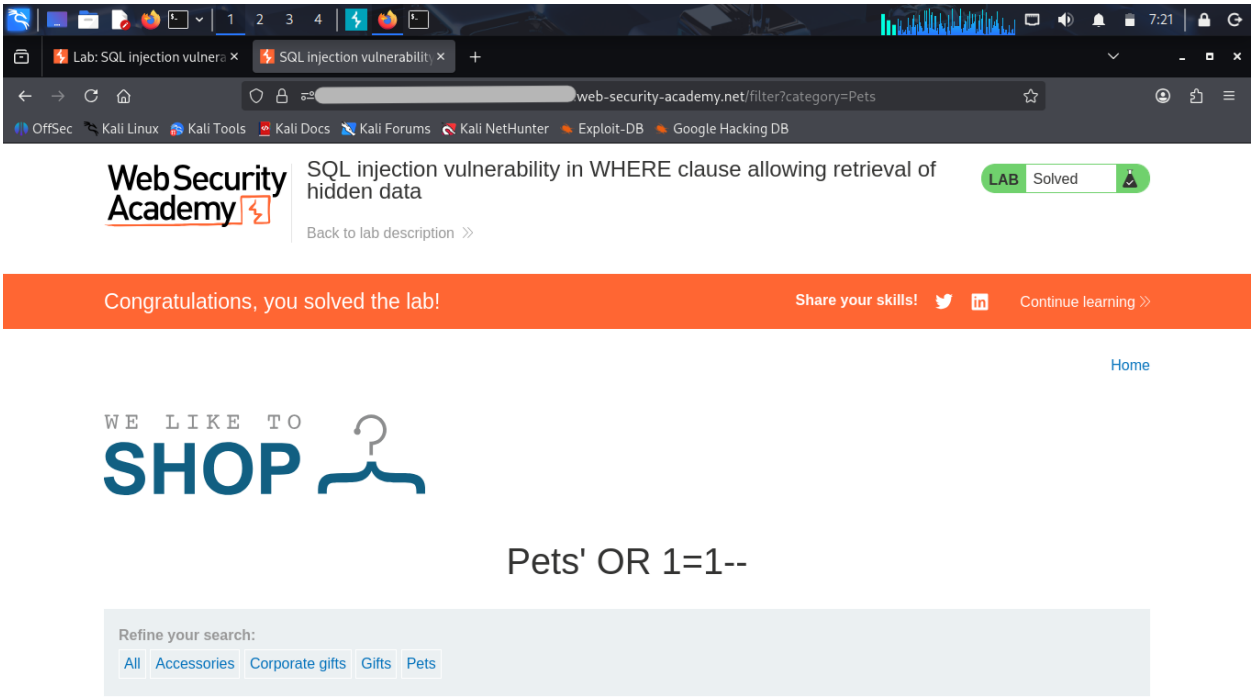


Figure 7: Lab environment confirming the challenge status as Solved.

4.3 Expected Result

The application should return only those products matching the selected category and marked as released, consistent with the intended query:

```
SELECT * FROM products WHERE category = 'Pets' AND released = 1
```

4.4 Observed Behavior

The application returned all product records, including those flagged as unreleased, as a result of the injected payload nullifying the released = 1 condition:

```
SELECT * FROM products WHERE category = 'Pets' OR 1=1--' AND released = 1
```

This confirms the presence of an exploitable SQL Injection vulnerability, permitting an attacker to bypass application-enforced data visibility controls.

5. Remediation

5.1 Primary Recommendation — Parameterized Queries

The application must replace dynamic SQL string concatenation with parameterized queries (prepared statements) for all database interactions involving user-supplied input. Parameterization ensures that input is always treated as literal data rather than executable SQL syntax, which eliminates this class of vulnerability at the root cause rather than mitigating its symptoms.

Example (illustrative):

```
SELECT * FROM products WHERE category = ? AND released = 1
```

The category value is passed as a bound parameter rather than being inserted into the query string, preventing the database from interpreting it as part of the query structure.

5.2 Secondary Recommendations — Defense in Depth

The following controls are recommended as supplementary layers of protection and should not be considered substitutes for parameterized queries:

- Least-privilege database accounts — The application's database account should be restricted to only the permissions required for its function (e.g., read-only access to the products table where applicable), limiting the impact of any future injection vulnerability.
- Input validation — Category values should be validated against an allow-list of expected values at the application layer prior to query execution.
- Web Application Firewall (WAF) — A WAF configured with SQL injection detection rules can provide an additional layer of monitoring and blocking for malicious payloads, serving as a compensating control.
- Error handling — Verbose database error messages should not be exposed to end users, as they can assist an attacker in refining injection payloads.

5.3 Verification

Following remediation, it is recommended that the fix be re-tested using the same payload documented in Section 4 to confirm that the released condition can no longer be bypassed, and that malformed or unexpected input is safely rejected or ignored by the query layer.

Conclusion

This assessment identified a High-severity SQL Injection vulnerability in the product category filtering functionality of the target application, resulting in unauthorized disclosure of unreleased product data. The finding was reproduced and validated within an authorized lab environment using a documented, repeatable methodology. It is recommended that the development team implement parameterized queries as the primary remediation, supported by the defense-in-depth controls outlined in Section 5, prior to any production deployment.